

MOATLY

Kickoff Pack

The Week-1 reference for the engineering lead. Pulls together the ADRs from all six technical breakdowns, lays out the team and hiring plan, and gives the lead a concrete Week-1 checklist.

Prepared for: Engineering lead (on day one)

Document version: 0.1 · Status: Draft for review

Date: April 2026

0. How to read this document

This is the **kickoff pack** — a short synthesis document that sits on top of six technical breakdowns and translates them into the work the engineering lead needs to do in Weeks 1–4 of the build. It is deliberately a reference rather than a narrative: use the table of contents, skim for what you need, don't read cover-to-cover.

The six breakdowns this pack summarises are:

- Moat App — Phase 1 (app-only MVP)
- Moat App — Phase 2 (Moats, Megolm, deletion client)
- Moat App — Phase 3 (device integration)
- Online Portal
- Backend (Moat service + linkage service)
- Firmware (nRF5340 / Zephyr)

Each of those documents stands alone; this pack does not repeat their content. What it does is *sequence* them — so that the lead knows what to read when, which decisions block which hires, and what 'good' looks like in the first four weeks.

0.1 Document map

§	Contents	When to use
1	The ADR pack — all decisions flagged across the six breakdowns, in one place.	Week 1, before the architecture review.
2	Team shape — roles, headcount, sequencing.	Before any hiring post goes out.
3	Hiring materials — sample job descriptions, technical-screen guidance.	As you begin hiring.
4	Week-1 checklist — day-by-day what-to-do for the lead.	Day 1.
5	The first 90 days — Phase 0 exit gate + what needs to be true.	Weeks 2 onwards.
6	Cross-cutting risks — the things that will bite in Months 1–3 if ignored early.	Always.
7	Communication — rituals, docs, what 'written down' means here.	Week 1.
8	Closing — the posture this project wants from its lead.	First, if you're starting cold.

1. The ADR pack

Architecture Decision Records (ADRs) are the durable record of choices made early that everything downstream depends on. The Build Brief (§C) listed 13 ADRs; the six breakdowns added another ~35 implementation-level decisions. This section collects them into one ratifiable list.

1.1 Product-level ADRs (from Build Brief)

ADR	Decision	Source
001	Use Olm/Megolm via vodozemacs (Apache 2.0) over libsignal (AGPL) or MLS.	Brief §C, §6.2
002	Go for Moat service; Go (or Node) for linkage/account service.	Brief §C, §4.2
003	React Native (Expo prebuild) for mobile (overruling the Flutter option).	Phase 1 §1.1
004	nRF5340 with single firmware image across Premium and Simple tiers.	Brief §C, §4.3
005	PostgreSQL 16 + Temporal for backend; no NoSQL.	Brief §C, §4.2
006	AWS EU-Frankfurt primary; Dublin failover.	Brief §C, §4.2
007	Apache 2.0 + Commons Clause for code; CC BY-NC-ND for content.	Brief §C, §12.3
008	Daily-rotated data encryption keys with 8-day life, KMS-wrapped.	Brief §C, §6.3
009	No third-party analytics SDKs — ever.	Brief §C, §2, §5.2
010	Device + combination + composite identity as the sole production auth path.	Brief §C, §5.5
011	Unity ritual app as sibling; opaque ID + email relay + shared tokens only.	Brief §C, §5.3
012	Lamp ships in every version from MVP, with in-bundle resource list.	Brief §C, §5.9
013	Complaint + reassignment operates on metadata only.	Brief §C, §5.10

1.2 Mobile app ADRs

ADR	Decision	Source
014	Expo prebuild (not Expo Go, not bare RN).	Phase 1 §1.2
015	Zustand + React Query for state; no Redux.	Phase 1 §1.3
016	React Navigation; no deep links into Moat screens.	Phase 1 §1.4
017	op-sqlite with SQLCipher; hand-written SQL, no ORM.	Phase 1 §1.5

ADR	Decision	Source
018	Email stored in expo-secure-store, separate from main DB.	Phase 1 §1.6
019	Jest + Maestro + fast-check for tests.	Phase 1 §1.7
020	UniFFI-generated vodozamac bindings + hand-written TS wrapper.	Phase 2 §1.1
021	Protobuf outer envelope + JSON inner body.	Phase 2 §1.2
022	Native WebSocket + hand-written reconnect (no socket.io).	Phase 2 §1.3
023	Server-authoritative cycle clock; client compares against trusted time.	Phase 2 §1.4
024	Server-assigned pseudonyms from a curated word-pair list; rotate per cycle.	Phase 2 §1.5
025	soft-auth hard-expires at Phase 3 go-live; not sliding.	Phase 2 §1.6
026	react-native-ble-plx with a telemetry-stripping wrapper.	Phase 3 §1.1
027	RSSI -70 dBm near / -85 dBm far, 3s debounce.	Phase 3 §1.2
028	Virtual device labelled 'reduced security' as persistent banner, not modal.	Phase 3 §1.3
029	In-box factory-provisioned attestation (not in-app).	Phase 3 §1.4
030	On-device-only combination verification; server never sees a combination hash.	Phase 3 §1.5
031	Timing-only behavioural fingerprint; 50ms buckets; no pressure data to backend.	Phase 3 §1.6

1.3 Backend ADRs

ADR	Decision	Source
032	net/http + chi for routing; no gin/echo/fiber.	Backend §2.1
033	sqlc for database layer; hand-written SQL, generated types, no ORM.	Backend §2.2
034	nhooyr (coder/websocket) for WebSocket; not gorilla (deprecated).	Backend §2.3
035	Temporal self-hosted in same cluster; not Temporal Cloud.	Backend §2.4
036	Server-side vodozamac parser-only; no decryption capability on server.	Backend §2.5
037	AWS KMS + SOPS for secrets.	Backend §2.6
038	Grafana Cloud (EU) for observability.	Backend §2.7
039	AWS ALB + WAF for load balancing.	Backend §2.8
040	RDS Postgres 16 Multi-AZ Frankfurt + Dublin read replica.	Backend §2.9

1.4 Portal ADRs

ADR	Decision	Source
041	Astro + MDX (not Next.js, not SvelteKit).	Portal §2.1
042	AWS EU Frankfurt for portal hosting (matches backend).	Portal §2.2
043	Separate portal Go service (not shared with Moat backend).	Portal §2.3
044	Stripe Checkout hosted (redirect-based); no Stripe.js on our domain.	Portal §2.4
045	Git-based MDX; no CMS.	Portal §2.5
046	Self-host every font and asset; no third-party CDNs.	Portal §2.6
047	Postmark transactional stream; no marketing stream ever.	Portal §2.7

1.5 Firmware ADRs

ADR	Decision	Source
048	Zephyr 3.7 LTS (not current).	Firmware §2.1
049	MCUboot for secure boot.	Firmware §2.4
050	Ed25519 for attestation; hardware-backed via PSA Crypto on CryptoCell-312.	Firmware §2.5
051	MCUboot + SMP for DFU (not Nordic legacy Secure DFU).	Firmware §2.6
052	Combination never stored; derives a PSA unlock credential via PBKDF2.	Firmware §2.7
053	Zero logging in production builds (subsystem compiled out).	Firmware §2.8
054	Tier detection via OTP byte (not hardware strap or flash flag).	Firmware §2.9
055	No BLE bonding; attestation + signed challenges are the security model.	Firmware §4.5

1.6 How to ratify this pack

One meeting. Lead + founder + security engineer (fractional, if hired). Two hours.

- Read the list aloud.
- For each ADR, one of three outcomes: ratify-as-default, ratify-with-modification, defer-pending-investigation.
- A deferred ADR becomes a Week-1 action item for whoever owns the investigation.
- A ratified-with-modification ADR generates an addendum; the original recommendation and the override are both recorded in the repo.
- The ADR pack, ratified, becomes the governance document for the project. Changing any ADR post-ratification requires founder sign-off and a written record of the reason.

2. Team shape

The Build Brief (§11.1) specifies the team composition. This section restates it with the sequencing that the breakdowns imply.

2.1 Full team to public beta

Role	FTE	Primary responsibility
Engineering lead / architect	1	System design, crypto review, ADR governance, cross-team coordination.
Mobile engineer (senior)	1	vodozamac FFI; crypto modules; session/deletion/device clusters.
Mobile engineer	1	UI layer; ritual-adjacent animation; pairing and combination UX.
Backend engineer (senior)	1	Moat service core; deletion pipeline; Temporal workflows.
Backend engineer	1	Linkage service; auth; WebSocket gateway; portal backend.
Firmware engineer	1	nRF5340 Zephyr firmware end-to-end.
Security engineer	0.5–1	Threat model; crypto review; audit liaison; ramps to full-time at Phase 3.
SRE / platform	1	Infra-as-code; observability; deletion-pipeline SLOs; deployment.
Designer	1	Ritual aesthetic; device industrial design liaison; micro-copy; design tokens.
Product / founder	1	Covenant editorial; cohort management; product-principle enforcement.

Total: ~9.5 FTE through public beta. Contracts to ~6 FTE post-launch. The *product* does not need more than this; the principles in §2 of the brief actively resist the engineering problem growing in ways that would need more.

2.2 Hiring sequence

Month	Hire	Why now
M1	Engineering lead	Before anything else is written. The lead ratifies the ADR pack.
M1 (½)	Security engineer (fractional)	The threat model must land before the first line of production code.
M2	Mobile senior + Mobile engineer	Phase 1 build team. Can onboard in parallel with Phase 0 infra work.
M2	Backend senior + Backend engineer	Same.
M2	Firmware engineer	Long lead time on hardware; cannot delay.

Month	Hire	Why now
M2	Designer	Works alongside founder from day one on ritual aesthetic + design tokens.
M3	SRE	Receives infra-as-code; sets up deletion-pipeline SLOs before Phase 2.
M7	Security engineer to full-time	Phase 3 device work + external audit engagement.

2.3 Who needs to talk to whom

Pair	Why they talk often	Cadence
Mobile senior + Backend senior	Every endpoint is a contract.	Daily during Phase 2.
Mobile senior + Firmware engineer	The BLE GATT profile is shared truth.	Weekly, Phase 3.
Security engineer + all engineers	Threat model is a living document.	Weekly.
Designer + Mobile engineer	Animation, tokens, micro-copy.	Daily during Phase 1 + early Phase 2.
Designer + Firmware engineer	Glow patterns, industrial design, hardware UX.	Weekly, Phase 3.
Founder + Engineering lead	Product-principle enforcement.	Weekly.
SRE + Backend senior	Deletion pipeline SLOs; incident response.	Weekly.

3. Hiring materials

3.1 What Moatly-ness looks like in an interview

Moatly's engineering culture is unusual in ways that are easy to misjudge at hire. The non-negotiable principles (brief §2) are not just rules — they are selection criteria. Candidates who are excited by 'no analytics, no profiles, no ranking' are self-selecting for the posture that makes the project possible. Candidates who see those as obstacles to shipping product features are a poor fit regardless of skill.

The Build Brief (§2.2) puts this directly: 'Feature requests that add observation, engagement, or retention mechanics will be rejected. A roadmap ticket that reads increase DAU will not be accepted.' An engineer who has not internalised this will burn out in six months.

3.2 Sample job description — mobile engineer

Moatly — Mobile Engineer (Senior, React Native + Rust FFI)

We are a small team building an unusual product: a privacy-first practice platform where messages are cryptographically destroyed every seven days and the hardware device is the authentication key. We are open source (Apache 2.0), EU-hosted, and have zero analytics anywhere in our stack — including in the mobile app. You will own the mobile client end-to-end: the Expo-prebuild React Native app, the op-sqlite/SQLCIPHER data layer, the UniFFI-based vodozemac integration, BLE pairing via react-native-ble-plx, and the device-authentication flow. You will write code that runs on every phone in every Moat. What we are looking for: strong TypeScript, real production React Native experience, comfort with Rust FFI (UniFFI or bindgen), and an instinct for code that is small and auditable. What matters more: a visible personal stance on privacy and an allergy to the 'just add one tracker' mindset. Bonus: familiarity with the Matrix ecosystem, Signal Protocol, or any end-to-end-encrypted messaging project. You will be compensated fairly. You will not be given a 'move fast and break things' mandate. You will help build something small, well, and for a long time.

3.3 Sample job description — firmware engineer

Moatly — Firmware Engineer (nRF5340 / Zephyr)

We are building a hand-sized hardware device that is the authentication key to a privacy-first practice platform. The device is cryptographically attested, has a hardware secure element (CryptoCell-312 on nRF5340), and runs Zephyr RTOS. Two hardware tiers (Premium with capacitive ring + RGB halo; Simple with single button + single LED) share one firmware image via runtime capability detection. You will own the firmware end-to-end: BLE GATT profile, PSA Crypto key management, the combination-verification engine, the glow driver, MCUboot-based signed DFU, power management to a 30-day battery target, and factory provisioning. What we are looking for: shipped Zephyr-on-nRF5x firmware; familiarity with PSA Crypto and secure elements; experience with BLE peripherals at production scale; comfort with MCUboot or equivalent signed boot; instinct for small, auditable, boringly correct C. What we are not looking for: novelty. This firmware is ~10k lines of C when done. It will be audited externally before launch. It will be open source. It needs to be right.

3.4 Sample job description — backend engineer (senior)

Moatly — Backend Engineer (Senior, Go)

We are building the server side of a privacy-first end-to-end-encrypted group messaging platform.

Two small Go services (a Moat service + a linkage service) deploy to AWS EU, hold no plaintext Moat content, and implement a verifiable seven-day deletion pipeline that is the product's most important engineering promise. You will own one of the two services or the deletion pipeline (depending on strengths). You will write Go that is auditable line-by-line: chi + sqlc + Temporal; no ORM; minimal dependencies. You will help design the deletion pipeline's Temporal workflows, the Megolm envelope parsing via vodozamac, the WebSocket fanout, and the PII-stripping logger that makes 'we hold no content' a CI-enforceable claim. What we are looking for: strong Go; real operational experience running a TLS-terminated HTTP service at scale; familiarity with cryptographic system design (not 'I built my own AES,' but 'I can read a threat model'); instinct for doing-less. This role will see a lot of code review. If you like shipping Go and defending your PRs in writing, you will be at home.

3.5 Technical screen — the Moatly signal

Standard technical interviews (reverse-a-string, system design a Twitter clone) give us the wrong signal for Moatly engineering. These questions do better:

- **For mobile candidates:** 'Here is the Moat App Phase 1 breakdown §4.11 — the body-stripping logger. Critique it. What would you do differently?'
- **For backend candidates:** 'Here is the Backend §6 deletion pipeline. Find a failure mode not listed. Explain how you'd detect it.'
- **For firmware candidates:** 'The combination engine (Firmware §5) stores an HMAC of the derived unlock key. Walk me through the threat model — what does this defend against, and what does it not?'
- **For everyone:** 'Imagine the founder asks you to add a single analytics event to measure how many users complete onboarding. What do you say?' (Good answer: 'No, because §2 principle 3 of the brief. Here is what I'd do instead — count the shape-only metric at the CDN log level with zero per-user data.')

3.6 Compensation posture

Pay market rates, no equity gimmicks, no 'mission discount.' The work is demanding; the principles are demanding; the compensation should not also be demanding. We are a small team working on something serious; we hire seriously.

4. Week-1 checklist (for the lead)

Day-by-day. This is what a new engineering lead should walk into and work through, assuming the founder has already hired them and the ratification meeting (§1.6) has happened.

4.1 Monday

- **Morning:** read the Build Brief in full. Do not skim. Everything else depends on this.
- **Morning:** read the ADR pack (§1 of this document).
- **Afternoon:** founder meeting. Ratify or defer every ADR. Record outcomes.
- **Afternoon:** set up the repo organisation on GitHub. Apache 2.0 + Commons Clause LICENSE in place. moatly-mobile, moatly-backend, moatly-portal, moatly-firmware as four separate repos.
- **End of day:** write a one-page kickoff note, email to founder. Confirms ADR ratification status and immediate blockers.

4.2 Tuesday

- **Morning:** read Phase 1 of the Moat App breakdown in full.
- **Afternoon:** read Phase 2 and Phase 3 — skim; you'll go back.
- **Afternoon:** identify the three decisions in each breakdown flagged 'needed by Week 1' that are not yet ratified. File them for founder decision this week.

4.3 Wednesday

- **Morning:** read the Backend breakdown in full. This is the longest and the most security-sensitive.
- **Afternoon:** write the threat-model doc v0.1 draft. Starts from the Build Brief §6 and adds any threats surfaced during the breakdowns' reads. This doc is what the security engineer will review first.

4.4 Thursday

- **Morning:** read the Portal and Firmware breakdowns.
- **Afternoon:** post job descriptions (§3). Four roles: mobile senior, mobile, backend senior, firmware.
- **Afternoon:** book a call with the external cryptographer (to be identified by founder or lead during the week).

4.5 Friday

- **Morning:** CI/CD baseline. GitHub Actions workflows for each repo: lint, unit, SAST, dependency audit, SBOM. Sample runs of each, even if the first commit is an empty scaffold.
- **Morning:** infrastructure-as-code scaffolding. One Terraform root module with an AWS EU account + a Dublin replica target. Not yet applied — just reviewed.
- **Afternoon:** write Week-1 retro. What landed? What didn't? What's blocking Week 2?
- **End of day:** email founder with Week-2 priorities. Ratify the priorities before Monday.

5. The first 90 days

Weeks 1–12 overlap with Phase 0 (foundations) and the start of Phase 1. The gate between them is specific and listed in brief §10.1.

5.1 Phase 0 exit gate

- ADRs 1–55 (per §1 above) approved by lead and founder.
- External cryptographer sign-off on the crypto design doc.
- First commit lands in the public repo.
- Infrastructure-as-code for staging and production defined.
- Threat model v1 reviewed and ratified.
- Covenant text v1 ratified by founder and legal.

5.2 Week-by-week high level

Week	Focus
1	Lead onboarding (this document). ADR ratification. Repos created. Job descriptions posted.
2–3	First engineers start. Threat model draft. Infrastructure-as-code applied to staging. External cryptographer engaged.
4–6	Phase 0 exit gate: crypto design review complete, ADRs approved, public repo live.
7–12	Phase 1 MVP build begins — app onboarding, Lamp, Two Questions, rings, Unity linkage. Parallel: portal scaffolded and first pages live.

5.3 Signals you are on track at day 90

- Ten people at the team. Mobile + backend + firmware + SRE + security (fractional) + designer + lead + founder.
- Phase 1 week-7 of the 12-week app plan is complete or nearly so. (Onboarding skeleton + Lamp + Two Questions on dogfood devices.)
- Portal has homepage, /how, /device pages live on staging.
- Firmware has booted on nRF5340 dev boards and can serve a stub GATT service.
- Backend linkage service running on staging; /v1/linkage/relay_prescription works end-to-end with a Postmark test send.
- First transparency-report template is in the portal repo (even if the numbers aren't meaningful yet).
- Threat model doc has been reviewed by the external cryptographer at least once.

5.4 Signals you are off track

- **You are spending time refactoring the ADR pack rather than shipping against it.** The ADR pack is a governance tool, not a design deliverable. If it is consuming week 6+, something is wrong.

- **Hiring is slow.** If you are still at 5 FTE at day 90, the 10-month timeline to public beta is no longer real.
- **A team is 'nearly done' with Phase 0 but hasn't run the crypto review.** The crypto review is a hard gate. 'Nearly done' without it is 'not done.'
- **The team is adding scope not in the brief.** Anything that reads 'just one more feature' should send the ADR pack back to the founder for explicit ratification before work starts.

6. Cross-cutting risks — surface these early

6.1 The vodozemac FFI is the technical critical path

The FFI from TypeScript to Rust is unprecedented on this team (likely). If it is not working by Phase 2 Week 3, the whole Moat-service phase slips. Mitigation: the senior mobile engineer spikes the FFI in Phase 1 as a pre-work item, not as a Phase 2 Week 1 task. If the spike fails, you have 6 weeks of Phase 1 to either solve it or escalate to 'fall back to hand-written Turbo Module.'

6.2 Hardware lead time

The brief (§14.1) says 'device industrial design slips; Phase 3 stalls' is 'High likelihood, high impact.' Every breakdown assumes dev boards are available by Week 1 of the firmware plan. If they are not, the firmware engineer starts two weeks behind and cascades into app Phase 3.

Mitigation: order dev boards immediately. Like, before your first engineer starts. Nordic nRF5340-DK is in stock at the usual distributors. Ten units at the lead, ten at the firmware engineer, ten spares for the bench. Order on day one.

6.3 The KMS minimum-deletion window

AWS KMS has a minimum scheduled-deletion window of 7 days. We target 8 days to give margin (brief §14.1, backend §6.4, ADR-008). This is a known constraint, not a problem — but if AWS changes this minimum or we discover another provider-level constraint, we have to redesign the backup encryption story. Mitigation: implement the daily-rotated-key pattern early in Phase 0, verify with AWS support that we understand the constraint correctly, document in ADR-008.

6.4 iOS silent-push throttling

The glow-to-read latency SLO (p95 <5s Android, <15s iOS) is hostage to iOS's silent-push behaviour. Brief §14.1 flags this. Mitigation: measure early with real devices; accept the p95 <15s bound on iOS; design the UX to feel calm even when the glow lags the post by several seconds.

6.5 Regulatory interpretation

UK Online Safety Act, EU DSA, Australian TOLA — all three have clauses that can be read as incompatible with an E2E-encrypted unmoderated platform. The Lamp and the complaint flow (brief §12.6) are our material compliance story. Mitigation: engage a regulatory-law specialist (UK + EU focus) during Phase 2; document the compliance story before the first public announcement.

6.6 Founder pressure for growth mechanics

Brief §14.1 lists this as 'Medium likelihood, Existential impact.' It is the most important risk on this list. Mitigation is not technical — it is written cultural. §8 of this document addresses it directly.

7. Communication

7.1 Rituals

Ritual	Cadence	What happens
Standup	Daily, async	Text channel. Each engineer posts: yesterday / today / blocked. No meetings.
Weekly team sync	Mon, 30 min	Cross-team updates, risks, hiring status. Facilitated by lead.
Fortnightly retrospective	90 min	What landed, what didn't, what to adjust. Open to whole team.
Monthly founder review	60 min	Lead + founder. Progress against roadmap, ADR changes, strategic questions.
Quarterly public transparency report	Public	Drafted by lead + SRE, reviewed by founder + legal. Published to the portal.
Incident post-mortem	On incident	Within 30 days of any incident. Public on the portal per brief §12.4.

7.2 Written culture

Moatly's engineering is **heavily written**. Decisions live in ADRs. Designs live in docs. Post-mortems are public. The six technical breakdowns are evidence of this posture — they exist so that a reader in Year 2 can understand why the codebase looks the way it looks. That culture starts in Week 1 or it never starts.

- Every non-trivial technical decision is an ADR.
- Every non-trivial scope expansion is a breakdown addendum.
- Every incident gets a post-mortem. Public, unedited.
- Every PR is reviewable in isolation; commit messages explain the why, not just the what.

7.3 What is not written

- User content. Ever. (The brief and breakdowns are emphatic.)
- Individual team members' performance evaluations — kept in a private HR system, as is normal.
- Pre-launch marketing copy in draft form — until the founder is ready to ship it.
- Unresolved legal matters — per legal counsel.

8. Closing

8.1 The posture this project wants from its lead

The lead of Moatly is not the lead of a growth-stage SaaS. The posture is different in four specific ways.

8.1.1 Cheerful refusal, not heroic defence

Every week, someone will ask for something that the principles in §2 of the brief say no to. The lead's job is to say no — cheerfully, briefly, with reference to the brief. Not with outrage. Not with long email replies. A three-sentence 'no, because §2 principle 3, here is an alternative that would work' is the entire ritual.

Heroic defence — long meetings, impassioned speeches, 'I cannot let this happen' email threads — is exhausting and counterproductive. The brief is load-bearing; it does the defending. The lead just has to point at it.

8.1.2 Slow, because the product is long-lived

Moatly is not a sprint. The product's value grows through time-in-practice, not feature velocity (brief §10.6). The engineering cadence should match. A feature that ships in six weeks instead of four is better than a feature that ships buggy and is patched for months. A codebase that reads clearly after two years is worth more than one that shipped two months earlier.

8.1.3 Boringly public

The codebase is open source. The transparency reports are public. The incident post-mortems are public. The architecture is publicly defensible in the sense that an external auditor can read everything and understand why.

This is a constraint and also a gift. It means decisions that would be informal in a closed company must be written down here. It also means the engineering team never has to do sales-driven work or investor-driven dashboards. The public record is the dashboard.

8.1.4 Aligned with the founder, not subservient to them

The founder is responsible for the product's editorial voice and commercial decisions. The lead is responsible for the engineering's correctness and the principles' enforcement. When they disagree — and they will — both parties owe each other a written account of the disagreement before either commits to action.

The lead does not *win* disagreements with the founder. The lead makes sure the disagreement is well-articulated, both sides are heard, and the outcome is written down. If the founder overrules the lead on a principle-adjacent matter, that is the founder's call — but the record shows it happened and why.

8.2 What you are not being asked to build

- You are not building the next billion-user platform.
- You are not chasing DAU or retention.
- You are not optimising a conversion funnel.
- You are not building a moat against competitors via feature velocity.
- You are not trying to get acquired.

8.3 What you are being asked to build

- A small product that takes its users' inner lives seriously.
- A codebase that can be publicly read and externally audited and will hold up.
- A system whose central engineering promise — cryptographic seven-day deletion — is verifiably true.
- A team that keeps the principles intact after the novelty wears off.
- A product that is around in ten years, not because it grew, but because it didn't stop.

8.4 Good luck

Every other document on your desk tells you what to build. This one tells you how to be the person building it. If the posture in §8.1 resonates, the rest of the plan follows. If it doesn't, Moatly might not be the right project, and that is its own useful information.

The engineering breakdowns are thorough. The ADRs are explicit. The team shape is set. The sequencing is laid out. What remains is to start — slowly, publicly, deliberately — and to keep going.

— *End of Kickoff Pack* —

Moatly · Kickoff pack · April 2026 · v0.1